

1. Zero-Shot Prompt

prompt-

Write synthesizable Verilog code for a 16-bit signed adder using STRICTLY STRUCTURAL MODELING ONLY.

STRICT RULE: The top-level adder module must be built EXCLUSIVELY by instantiating sub-modules and connecting them with wires. No assign statements, no always blocks, no arithmetic operators (+, -, etc.) are allowed inside the top-level structural module. All logic must live inside primitive sub-modules (full_adder, half_adder, etc.) which are then wired together in the top-level module.

The design must:

- Define a 1-bit full_adder module using gate primitives (and, or, xor) or assign inside it
- Instantiate exactly 16 full_adder modules to form a 16-bit ripple carry adder
- Accept two 16-bit signed inputs: A and B
- Produce a 17-bit signed output: sum[16:0] where sum[16] captures the final carry out
- Wire the carry chain explicitly: cout of bit N connects to cin of bit N+1
- Handle signed two's complement correctly

Also write a testbench that:

- Tests ALL corner cases using a for loop over 20 boundary value pairs
- Uses a while loop with \$signed(\$random) for 100 iterations
- Uses a repeat loop for 50 random iterations
- Uses a forever loop for clock toggling stopped by \$finish at 1000ns
- Checks results using \$display with signed decimal format
- Prints PASS or FAIL per test

Design module-

Structural Full Adder -

```
module full_adder (  
    input a,  
    input b,  
    input cin,  
    output sum,  
    output cout  
);
```

```
    wire x1;  
    wire a1, a2, a3;
```

```
    xor (x1, a, b);  
    xor (sum, x1, cin);
```

```
    and (a1, a, b);  
    and (a2, a, cin);  
    and (a3, b, cin);
```

```
    or (cout, a1, a2, a3);
```

```
endmodule
```

16-bit Signed Adder (STRICTLY STRUCTURAL) -

```
module signed_adder_16_structural (
```

```
    input signed [15:0] A,
```

```
    input signed [15:0] B,
```

```
    output signed [16:0] sum
```

```
);
```

```
    wire c1, c2, c3, c4, c5, c6, c7, c8;
```

```
    wire c9, c10, c11, c12, c13, c14, c15, c16;
```

```
    full_adder FA0 (
```

```
        .a (A[0]),
```

```
        .b (B[0]),
```

```
        .cin (1'b0),
```

```
        .sum (sum[0]),
```

```
        .cout(c1)
```

```
    );
```

```
    full_adder FA1 (
```

```
        .a (A[1]),
```

```
        .b (B[1]),
```

```
        .cin (c1),
```

```
        .sum (sum[1]),
```

```
        .cout(c2)
```

```
    );
```

```
    full_adder FA2 (
```

```
        .a (A[2]),
```

```
        .b (B[2]),
```

```
        .cin (c2),
```

```
        .sum (sum[2]),
```

```
        .cout(c3)
```

```
    );
```

```
    full_adder FA3 (
```

```
        .a (A[3]),
```

```
        .b (B[3]),
```

```
        .cin (c3),
```

```
        .sum (sum[3]),
```

```
        .cout(c4)
```

```
    );
```

```
    full_adder FA4 (
```

```
        .a (A[4]),
```

```
.b (B[4]),  
.cin (c4),  
.sum (sum[4]),  
.cout(c5)  
);
```

```
full_adder FA5 (  
.a (A[5]),  
.b (B[5]),  
.cin (c5),  
.sum (sum[5]),  
.cout(c6)  
);
```

```
full_adder FA6 (  
.a (A[6]),  
.b (B[6]),  
.cin (c6),  
.sum (sum[6]),  
.cout(c7)  
);
```

```
full_adder FA7 (  
.a (A[7]),  
.b (B[7]),  
.cin (c7),  
.sum (sum[7]),  
.cout(c8)  
);
```

```
full_adder FA8 (  
.a (A[8]),  
.b (B[8]),  
.cin (c8),  
.sum (sum[8]),  
.cout(c9)  
);
```

```
full_adder FA9 (  
.a (A[9]),  
.b (B[9]),  
.cin (c9),  
.sum (sum[9]),  
.cout(c10)  
);
```

```
full_adder FA10 (  
.a (A[10]),
```

```
.b (B[10]),  
.cin (c10),  
.sum (sum[10]),  
.cout(c11)  
);
```

```
full_adder FA11 (  
.a (A[11]),  
.b (B[11]),  
.cin (c11),  
.sum (sum[11]),  
.cout(c12)  
);
```

```
full_adder FA12 (  
.a (A[12]),  
.b (B[12]),  
.cin (c12),  
.sum (sum[12]),  
.cout(c13)  
);
```

```
full_adder FA13 (  
.a (A[13]),  
.b (B[13]),  
.cin (c13),  
.sum (sum[13]),  
.cout(c14)  
);
```

```
full_adder FA14 (  
.a (A[14]),  
.b (B[14]),  
.cin (c14),  
.sum (sum[14]),  
.cout(c15)  
);
```

```
full_adder FA15 (  
.a (A[15]),  
.b (B[15]),  
.cin (c15),  
.sum (sum[15]),  
.cout(c16)  
);
```

```
buf (sum[16], c16);
```

endmodule

2. Few-Shot Prompt

Prompt-

I will show you examples of structural Verilog modeling. You will extend them to a full 16-bit signed adder with testbench.

STRICT RULE: The top-level adder module must be built ONLY by instantiating sub-modules and wiring them together. No assign, no always, no arithmetic operators inside the top-level structural module. Logic only lives inside primitive sub-modules.

EXAMPLE 1 — 1-bit half adder (primitive level):

```
module half_adder(input a, b, output sum, cout);
  xor g1(sum, a, b);
  and g2(cout, a, b);
endmodule
```

EXAMPLE 2 — 1-bit full adder using structural instantiation:

```
module full_adder(input a, b, cin, output sum, cout);
  wire s1, c1, c2;
  half_adder ha1(.a(a), .b(b), .sum(s1), .cout(c1));
  half_adder ha2(.a(s1), .b(cin), .sum(sum), .cout(c2));
  or g1(cout, c1, c2);
endmodule
```

EXAMPLE 3 — 4-bit structural ripple carry adder:

```
module adder4_struct(input [3:0] A, B, input cin, output [3:0] sum, output cout);
  wire [4:0] c;
  assign c[0] = cin;
  full_adder fa0(.a(A[0]), .b(B[0]), .cin(c[0]), .sum(sum[0]), .cout(c[1]));
  full_adder fa1(.a(A[1]), .b(B[1]), .cin(c[1]), .sum(sum[1]), .cout(c[2]));
  full_adder fa2(.a(A[2]), .b(B[2]), .cin(c[2]), .sum(sum[2]), .cout(c[3]));
  full_adder fa3(.a(A[3]), .b(B[3]), .cin(c[3]), .sum(sum[3]), .cout(c[4]));
  assign cout = c[4];
endmodule
```

NOW DO THE FOLLOWING:

1. Use EXAMPLE 1 as your half_adder primitive.
2. Use EXAMPLE 2 as your full_adder built from half_adder instances.
3. Scale EXAMPLE 3 to a 16-bit signed adder (adder16_struct):
 - Instantiate exactly 16 full_adder modules (fa0 through fa15)
 - Wire carry chain: c[0] through c[16]
 - c[0] = 0 (no initial carry for addition)
 - Output: wire the 16 sum bits + c[16] as the 17th bit (overflow)
 - Declare all ports as signed
4. Build a testbench with ALL four loop types:

- for loop: 20 boundary corner case pairs
 - while loop: 100 iterations with \$signed(\$random) inputs
 - repeat loop: 50 iterations with random inputs
 - forever loop: clock toggle every 5ns, \$finish at 1000ns
- Include PASS/FAIL checking and \$display in every loop.

Design module-

```
`timescale 1ns/1ps
```

```
//=====
// Half Adder Primitive
//=====
module half_adder(
input a,
input b,
output sum,
output cout
);
xor g1(sum, a, b);
and g2(cout, a, b);
endmodule

//=====
// Full Adder (Structural)
//=====
module full_adder(
input a,
input b,
input cin,
output sum,
output cout
);
wire s1;
wire c1;
wire c2;

half_adder ha1(
.a(a),
.b(b),
.sum(s1),
.cout(c1)
);

half_adder ha2(
.a(s1),
.b(cin),
.sum(sum),
.cout(c2)
);
```

```

);

or g1(cout, c1, c2);

endmodule

//=====
// 16-bit Signed Structural Ripple Carry Adder
//=====
module adder16_struct(
input signed [15:0] A,
input signed [15:0] B,
output signed [16:0] SUM
);

wire [16:0] c;

supply0 GND;

full_adder fa0 (
.a(A[0]), .b(B[0]), .cin(GND),
.sum(SUM[0]), .cout(c[1])
);

full_adder fa1 (
.a(A[1]), .b(B[1]), .cin(c[1]),
.sum(SUM[1]), .cout(c[2])
);

full_adder fa2 (
.a(A[2]), .b(B[2]), .cin(c[2]),
.sum(SUM[2]), .cout(c[3])
);

full_adder fa3 (
.a(A[3]), .b(B[3]), .cin(c[3]),
.sum(SUM[3]), .cout(c[4])
);

full_adder fa4 (
.a(A[4]), .b(B[4]), .cin(c[4]),
.sum(SUM[4]), .cout(c[5])
);

full_adder fa5 (
.a(A[5]), .b(B[5]), .cin(c[5]),
.sum(SUM[5]), .cout(c[6])
);

```

```
full_adder fa6 (  
  .a(A[6]), .b(B[6]), .cin(c[6]),  
  .sum(SUM[6]), .cout(c[7])  
);  
  
full_adder fa7 (  
  .a(A[7]), .b(B[7]), .cin(c[7]),  
  .sum(SUM[7]), .cout(c[8])  
);  
  
full_adder fa8 (  
  .a(A[8]), .b(B[8]), .cin(c[8]),  
  .sum(SUM[8]), .cout(c[9])  
);  
  
full_adder fa9 (  
  .a(A[9]), .b(B[9]), .cin(c[9]),  
  .sum(SUM[9]), .cout(c[10])  
);  
  
full_adder fa10 (  
  .a(A[10]), .b(B[10]), .cin(c[10]),  
  .sum(SUM[10]), .cout(c[11])  
);  
  
full_adder fa11 (  
  .a(A[11]), .b(B[11]), .cin(c[11]),  
  .sum(SUM[11]), .cout(c[12])  
);  
  
full_adder fa12 (  
  .a(A[12]), .b(B[12]), .cin(c[12]),  
  .sum(SUM[12]), .cout(c[13])  
);  
  
full_adder fa13 (  
  .a(A[13]), .b(B[13]), .cin(c[13]),  
  .sum(SUM[13]), .cout(c[14])  
);  
  
full_adder fa14 (  
  .a(A[14]), .b(B[14]), .cin(c[14]),  
  .sum(SUM[14]), .cout(c[15])  
);  
  
full_adder fa15 (  
  .a(A[15]), .b(B[15]), .cin(c[15]),
```



```
.sum(SUM[15]), .cout(SUM[16])  
);
```

```
endmodule
```

3. Chain-of-Thought Prompt =

Prompt-

Think step by step and reason through each decision before writing Verilog code.

STRICT RULE: The top-level adder module must be built ONLY through sub-module instantiation

and wire connections. No assign, no always, no arithmetic operators inside the top-level module.

All combinational logic must live inside primitive sub-modules only.

STEP 1 — UNDERSTAND STRUCTURAL MODELING:

Think: What is structural modeling in Verilog? It is building a module ONLY by instantiating other modules and connecting their ports via wires. The top-level module has NO logic of its own.

Why is it useful? It maps directly to physical gate-level netlists used in FPGA synthesis.

STEP 2 — DESIGN THE PRIMITIVE: half_adder

Think: What is the logic of a half adder?

sum = a XOR b

cout = a AND b

Write a half_adder module using Verilog gate primitives (xor, and).

STEP 3 — DESIGN THE FULL ADDER: full_adder

Think: How is a full adder built from two half adders?

First half adder: s1 = a XOR b, c1 = a AND b

Second half adder: sum = s1 XOR cin, c2 = s1 AND cin

cout = c1 OR c2

Write full_adder by instantiating two half_adder modules and one or gate. No assign inside.

STEP 4 — DESIGN THE 16-BIT STRUCTURAL ADDER: adder16_struct

Think: How do 16 full adders chain together?

- Declare wire [16:0] c for the carry chain

- c[0] = 1'b0 (no initial carry)

- fa0: A[0], B[0], c[0] → sum[0], c[1]

- fa1: A[1], B[1], c[1] → sum[1], c[2]

- ... continue through fa15 ...

- fa15: A[15], B[15], c[15] → sum[15], c[16]

- Final output sum[16] = c[16] (overflow/carry bit)

Write the module with 16 explicit full_adder instantiations. No assign except c[0]=1'b0.

STEP 5 — DESIGN THE TESTBENCH:

Think: What are ALL corner cases for 16-bit signed addition?

(0,0), (32767,0), (-32768,0), (32767,1), (-32768,-1),

(32767,32767), (-32768,-32768), (32767,-32768), (-1,1), etc.

Write testbench with:

- for loop: 20 explicit corner case pairs, #10 delay, PASS/FAIL check
- while loop: 100 iterations, \$signed(\$random) inputs, mismatch tracking
- repeat loop: 50 iterations, \$signed(\$random) inputs, display results
- forever loop: clk toggle every 5ns, \$finish at 1000ns

STEP 6 — VERIFY:

Re-check: Does adder16_struct contain ONLY module instantiations and wires?
Is signed on all ports? Is output 17 bits? Are all 4 loops present? Fix any issues.

Design module-

```
`timescale 1ns/1ps
```

```
//=====
```

```
// Half Adder
```

```
//=====
```

```
module half_adder(
```

```
input a,
```

```
input b,
```

```
output sum,
```

```
output cout
```

```
);
```

```
xor (sum, a, b);
```

```
and (cout, a, b);
```

```
endmodule
```

```
//=====
```

```
// Full Adder
```

```
// Built structurally from two half adders
```

```
//=====
```

```
module full_adder(
```

```
input a,
```

```
input b,
```

```
input cin,
```

```
output sum,
```

```
output cout
```

```
);
```

```
wire s1;
```

```
wire c1;
```

```
wire c2;
```

```
half_adder ha1 (
```

```
.a(a),
```

```

.b(b),
.sum(s1),
.cout(c1)
);

half_adder ha2 (
.a(s1),
.b(cin),
.sum(sum),
.cout(c2)
);

or (cout, c1, c2);

endmodule

```

```

//=====================================================
// 16-Bit Signed Structural Adder
// Top level contains ONLY module instantiations
// and wire connections.
//=====================================================
module adder16_struct(
input signed [15:0] A,
input signed [15:0] B,
output signed [16:0] SUM
);

wire [15:1] c;

full_adder fa0 (
.a (A[0]),
.b (B[0]),
.cin (1'b0),
.sum (SUM[0]),
.cout(c[1])
);

full_adder fa1 (
.a (A[1]),
.b (B[1]),
.cin (c[1]),
.sum (SUM[1]),
.cout(c[2])
);

full_adder fa2 (
.a (A[2]),

```

```
.b (B[2]),  
.cin (c[2]),  
.sum (SUM[2]),  
.cout(c[3])  
);
```

```
full_adder fa3 (  
.a (A[3]),  
.b (B[3]),  
.cin (c[3]),  
.sum (SUM[3]),  
.cout(c[4])  
);
```

```
full_adder fa4 (  
.a (A[4]),  
.b (B[4]),  
.cin (c[4]),  
.sum (SUM[4]),  
.cout(c[5])  
);
```

```
full_adder fa5 (  
.a (A[5]),  
.b (B[5]),  
.cin (c[5]),  
.sum (SUM[5]),  
.cout(c[6])  
);
```

```
full_adder fa6 (  
.a (A[6]),  
.b (B[6]),  
.cin (c[6]),  
.sum (SUM[6]),  
.cout(c[7])  
);
```

```
full_adder fa7 (  
.a (A[7]),  
.b (B[7]),  
.cin (c[7]),  
.sum (SUM[7]),  
.cout(c[8])  
);
```

```
full_adder fa8 (  
.a (A[8]),
```

```
.b (B[8]),  
.cin (c[8]),  
.sum (SUM[8]),  
.cout(c[9])  
);
```

```
full_adder fa9 (  
.a (A[9]),  
.b (B[9]),  
.cin (c[9]),  
.sum (SUM[9]),  
.cout(c[10])  
);
```

```
full_adder fa10 (  
.a (A[10]),  
.b (B[10]),  
.cin (c[10]),  
.sum (SUM[10]),  
.cout(c[11])  
);
```

```
full_adder fa11 (  
.a (A[11]),  
.b (B[11]),  
.cin (c[11]),  
.sum (SUM[11]),  
.cout(c[12])  
);
```

```
full_adder fa12 (  
.a (A[12]),  
.b (B[12]),  
.cin (c[12]),  
.sum (SUM[12]),  
.cout(c[13])  
);
```

```
full_adder fa13 (  
.a (A[13]),  
.b (B[13]),  
.cin (c[13]),  
.sum (SUM[13]),  
.cout(c[14])  
);
```

```
full_adder fa14 (  
.a (A[14]),
```

```
.b (B[14]),
.cin (c[14]),
.sum (SUM[14]),
.cout(c[15])
);
```

```
full_adder fa15 (
.a (A[15]),
.b (B[15]),
.cin (c[15]),
.sum (SUM[15]),
.cout(SUM[16])
);
```

```
endmodule
```

4. Role Prompting

Prompt-

You are a senior RTL design engineer specializing in gate-level and structural Verilog for FPGA synthesis and ASIC standard cell mapping. Your team enforces a strict rule: top-level RTL modules must be written using ONLY structural modeling — sub-module instantiation and wire connections. No assign statements, no always blocks, and no arithmetic operators are permitted inside any top-level structural module. All logic must be encapsulated inside primitive sub-modules.

A junior engineer has asked you to write a reference implementation of a 16-bit signed adder that strictly follows your team's structural-only coding standard.

Write the following as a professional code review submission:

1. Module half_adder (gate primitive level):
 - Inputs: a, b
 - Outputs: sum, cout
 - Use Verilog gate primitives: xor, and
 - No assign, no always
2. Module full_adder (structural, built from half_adder):
 - Inputs: a, b, cin
 - Outputs: sum, cout
 - Instantiate two half_adder modules and one or gate
 - No assign, no always, no arithmetic operators
3. Module adder16_struct (structural top-level):
 - Inputs: signed [15:0] A, B
 - Output: signed [16:0] sum
 - Instantiate exactly 16 full_adder modules named fa0 through fa15
 - Declare wire [16:0] c for carry chain

- Wire c[0] to ground (1'b0)
- Connect carry chain: c[1] through c[16]
- sum[15:0] from full adder outputs, sum[16] = c[16]
- NO assign, NO always, NO arithmetic operators inside this module

4. Testbench tb_adder16:

- FOR loop: 20 boundary pairs including all overflow corner cases with PASS/FAIL
- WHILE loop: 100 iterations with \$signed(\$random), error count tracking
- REPEAT loop: 50 iterations with random inputs, display each result
- FOREVER loop: clk toggle every 5ns, \$finish at 2000ns
- \$display with %0d signed decimal format throughout

Write clean, professional, commented Verilog with header comment blocks.

Design module-

```
//=====
=====
// File : adder16_struct.v
// Description : Strict Structural 16-bit Signed Adder Reference Implementation
// Author : RTL Design Team
//
// Coding Rules:
// - Top-level module uses ONLY structural modeling.
// - No assign statements in top-level.
// - No always blocks in top-level.
// - No arithmetic operators in top-level.
// - Logic is encapsulated inside primitive sub-modules.
//
// Modules:
// 1. half_adder
// 2. full_adder
// 3. adder16_struct
// 4. tb_adder16
//=====
=====

`timescale 1ns/1ps

//=====
=====
// HALF ADDER
// Primitive Gate-Level Implementation
//=====
=====
module half_adder (
input a,
input b,
output sum,
```

```
output cout  
);
```

```
xor (sum, a, b);  
and (cout, a, b);
```

```
endmodule
```

```
//=====
```

```
=====
```

```
// FULL ADDER
```

```
// Structural Implementation Using Two Half Adders
```

```
//=====
```

```
=====
```

```
module full_adder (  
input a,  
input b,  
input cin,  
output sum,  
output cout  
);  
  
wire s1;  
wire c1;  
wire c2;  
  
half_adder ha1 (  
.a (a),  
.b (b),  
.sum (s1),  
.cout(c1)  
);  
  
half_adder ha2 (  
.a (s1),  
.b (cin),  
.sum (sum),  
.cout(c2)  
);  
  
or g1 (  
cout,  
c1,  
c2  
);  
  
endmodule
```



```

//=====
=====
// 16-BIT STRUCTURAL SIGNED ADDER
//
// Structural only:
// - Exactly 16 full adder instances (fa0 ... fa15)
// - Carry chain c[0] -> c[16]
// - No assign
// - No always
// - No arithmetic operators
//=====
=====
module adder16_struct (
input signed [15:0] A,
input signed [15:0] B,
output signed [16:0] sum
);

wire [16:0] c;

supply0 GND;

// Carry-in = 0
buf (c[0], GND);

full_adder fa0 (
.a (A[0]),
.b (B[0]),
.cin (c[0]),
.sum (sum[0]),
.cout(c[1])
);

full_adder fa1 (
.a (A[1]),
.b (B[1]),
.cin (c[1]),
.sum (sum[1]),
.cout(c[2])
);

full_adder fa2 (
.a (A[2]),
.b (B[2]),
.cin (c[2]),
.sum (sum[2]),

```

```
.cout(c[3])  
);
```

```
full_adder fa3 (  
.a (A[3]),  
.b (B[3]),  
.cin (c[3]),  
.sum (sum[3]),  
.cout(c[4])  
);
```

```
full_adder fa4 (  
.a (A[4]),  
.b (B[4]),  
.cin (c[4]),  
.sum (sum[4]),  
.cout(c[5])  
);
```

```
full_adder fa5 (  
.a (A[5]),  
.b (B[5]),  
.cin (c[5]),  
.sum (sum[5]),  
.cout(c[6])  
);
```

```
full_adder fa6 (  
.a (A[6]),  
.b (B[6]),  
.cin (c[6]),  
.sum (sum[6]),  
.cout(c[7])  
);
```

```
full_adder fa7 (  
.a (A[7]),  
.b (B[7]),  
.cin (c[7]),  
.sum (sum[7]),  
.cout(c[8])  
);
```

```
full_adder fa8 (  
.a (A[8]),  
.b (B[8]),  
.cin (c[8]),  
.sum (sum[8]),
```

```
.cout(c[9])  
);
```

```
full_adder fa9 (  
.a (A[9]),  
.b (B[9]),  
.cin (c[9]),  
.sum (sum[9]),  
.cout(c[10])  
);
```

```
full_adder fa10 (  
.a (A[10]),  
.b (B[10]),  
.cin (c[10]),  
.sum (sum[10]),  
.cout(c[11])  
);
```

```
full_adder fa11 (  
.a (A[11]),  
.b (B[11]),  
.cin (c[11]),  
.sum (sum[11]),  
.cout(c[12])  
);
```

```
full_adder fa12 (  
.a (A[12]),  
.b (B[12]),  
.cin (c[12]),  
.sum (sum[12]),  
.cout(c[13])  
);
```

```
full_adder fa13 (  
.a (A[13]),  
.b (B[13]),  
.cin (c[13]),  
.sum (sum[13]),  
.cout(c[14])  
);
```

```
full_adder fa14 (  
.a (A[14]),  
.b (B[14]),  
.cin (c[14]),  
.sum (sum[14]),
```

```
.cout(c[15])  
);
```

```
full_adder fa15 (  
.a (A[15]),  
.b (B[15]),  
.cin (c[15]),  
.sum (sum[15]),  
.cout(c[16])  
);
```

```
// MSB carry-out becomes bit[16] of result  
buf (sum[16], c[16]);
```

```
endmodule
```

5. Instruction + Format Prompt - Prompt-

TASK: Write a 16-bit signed adder in Verilog using STRICTLY STRUCTURAL MODELING ONLY.

ABSOLUTE RULE: The top-level structural module adder16_struct must contain ONLY:

- wire declarations
- module instantiations with named port connections

NO assign statements (except grounding `c[0] = 1'b0` via tie-off instantiation or wire assignment for carry init)

NO always blocks

NO arithmetic operators (+, -, &, |, etc.) inside the top-level module

ALL logic must be inside primitive sub-modules

OUTPUT FORMAT — deliver in exactly this structure:

--- SECTION 1: HALF ADDER PRIMITIVE ---

Module: half_adder

Ports: input a, b | output sum, cout

Implementation: gate primitives only (xor, and)

[Write Verilog here]

--- SECTION 2: FULL ADDER MODULE ---

Module: full_adder

Ports: input a, b, cin | output sum, cout

Implementation: two half_adder instances + one or gate

[Write Verilog here]

--- SECTION 3: 16-BIT STRUCTURAL ADDER ---

Module: adder16_struct

Ports: input signed [15:0] A, B | output signed [16:0] sum

Implementation:

- wire [16:0] c (carry chain)
- c[0] tied to 1'b0
- fa0 through fa15: 16 full_adder instances
- sum[15:0] from fa outputs, sum[16] = c[16]

[Write Verilog here — NO assign, NO always, NO arithmetic operators]

--- SECTION 4: TESTBENCH ---

Module: tb_adder16

Instantiate: adder16_struct

FOR LOOP — test exactly these 20 input pairs (A, B):

(0,0), (1,0), (0,1), (-1,0), (0,-1), (1,1), (-1,-1), (1,-1),
(32767,0), (-32768,0), (32767,1), (-32768,-1),
(32767,32767), (-32768,-32768), (32767,-32768), (-32768,32767),
(16384,16384), (-16384,-16384), (32766,1), (-32767,-1)

Per iteration: apply → #10 → display A, B, expected, actual → PASS or FAIL

WHILE LOOP: test_count < 100, \$signed(\$random) inputs, error_count tracking

REPEAT LOOP: 50 iterations, \$signed(\$random) inputs, display each

FOREVER LOOP: clk toggle every 5ns, \$finish at 2000ns in separate initial block

DECLARATIONS:

reg signed [15:0] A, B

wire signed [16:0] sum

Use %0d in all \$display statements

Design module-

```
module half_adder(  
    input a,  
    input b,  
    output sum,  
    output cout  
);
```

```
    xor (sum, a, b);  
    and (cout, a, b);
```

```
endmodule
```

```
module full_adder(  
    input a,  
    input b,  
    input cin,  
    output sum,  
    output cout  
);
```

```
wire s1;  
wire c1;  
wire c2;
```

```
half_adder ha1 (  
    .a (a),  
    .b (b),  
    .sum (s1),  
    .cout(c1)  
);
```

```
half_adder ha2 (  
    .a (s1),  
    .b (cin),  
    .sum (sum),  
    .cout(c2)  
);
```

```
or (cout, c1, c2);
```

```
endmodule
```

```
module adder16_struct(  
    input signed [15:0] A,  
    input signed [15:0] B,  
    output signed [16:0] sum  
);
```

```
wire [16:0] c;
```

```
assign c[0] = 1'b0;
```

```
full_adder fa0 (  
    .a (A[0]),  
    .b (B[0]),  
    .cin (c[0]),  
    .sum (sum[0]),  
    .cout(c[1])  
);
```

```
full_adder fa1 (  
    .a (A[1]),  
    .b (B[1]),  
    .cin (c[1]),  
    .sum (sum[1]),  
    .cout(c[2])  
);
```

```
full_adder fa2 (  
    .a (A[2]),  
    .b (B[2]),  
    .cin (c[2]),  
    .sum (sum[2]),  
    .cout(c[3])  
);
```

```
full_adder fa3 (  
    .a (A[3]),  
    .b (B[3]),  
    .cin (c[3]),  
    .sum (sum[3]),  
    .cout(c[4])  
);
```

```
full_adder fa4 (  
    .a (A[4]),  
    .b (B[4]),  
    .cin (c[4]),  
    .sum (sum[4]),  
    .cout(c[5])  
);
```

```
full_adder fa5 (  
    .a (A[5]),  
    .b (B[5]),  
    .cin (c[5]),  
    .sum (sum[5]),  
    .cout(c[6])  
);
```

```
full_adder fa6 (  
    .a (A[6]),  
    .b (B[6]),  
    .cin (c[6]),  
    .sum (sum[6]),  
    .cout(c[7])  
);
```

```
full_adder fa7 (  
    .a (A[7]),  
    .b (B[7]),  
    .cin (c[7]),  
    .sum (sum[7]),  
    .cout(c[8])  
);
```

```
full_adder fa8 (  
    .a (A[8]),  
    .b (B[8]),  
    .cin (c[8]),  
    .sum (sum[8]),  
    .cout(c[9])  
);
```

```
full_adder fa9 (  
    .a (A[9]),  
    .b (B[9]),  
    .cin (c[9]),  
    .sum (sum[9]),  
    .cout(c[10])  
);
```

```
full_adder fa10 (  
    .a (A[10]),  
    .b (B[10]),  
    .cin (c[10]),  
    .sum (sum[10]),  
    .cout(c[11])  
);
```

```
full_adder fa11 (  
    .a (A[11]),  
    .b (B[11]),  
    .cin (c[11]),  
    .sum (sum[11]),  
    .cout(c[12])  
);
```

```
full_adder fa12 (  
    .a (A[12]),  
    .b (B[12]),  
    .cin (c[12]),  
    .sum (sum[12]),  
    .cout(c[13])  
);
```

```
full_adder fa13 (  
    .a (A[13]),  
    .b (B[13]),  
    .cin (c[13]),  
    .sum (sum[13]),  
    .cout(c[14])  
);
```



```

full_adder fa14 (
    .a (A[14]),
    .b (B[14]),
    .cin (c[14]),
    .sum (sum[14]),
    .cout(c[15])
);

full_adder fa15 (
    .a (A[15]),
    .b (B[15]),
    .cin (c[15]),
    .sum (sum[15]),
    .cout(c[16])
);

buf (sum[16], c[16]);

endmodule

```

6. Negative Prompting

Prompt-

Write a 16-bit signed adder in Verilog using structural modeling only, with a comprehensive testbench.

STRICT STRUCTURAL ENFORCEMENT — DO NOT violate any of these:

- DO NOT use assign statements inside the top-level adder16_struct module.
- DO NOT use always blocks anywhere in the design modules.
- DO NOT use arithmetic operators (+, -, &, |, ^) inside the top-level structural module.
- DO NOT write a single-module adder — you must have at least three modules: half_adder, full_adder, and adder16_struct.
- DO NOT use a generate loop to instantiate full adders — write all 16 instantiations explicitly (fa0 through fa15) so the structural hierarchy is clearly visible.
- DO NOT skip the carry chain — wire [16:0] c must be declared and connected bit by bit.
- DO NOT leave c[0] floating — it must be tied to 1'b0.
- DO NOT use only 16 output bits — sum must be signed [16:0] with sum[16] = c[16].
- DO NOT omit the signed keyword on ports in adder16_struct.
- DO NOT write a testbench with only one loop type — include for, while, repeat, and forever.
- DO NOT omit corner cases — explicitly test: 32767+1, -32768+(-1), 32767+32767, -32768+(-32768).
- DO NOT use \$random without \$signed() cast — use \$signed(\$random) for signed test values.
- DO NOT skip #delay between applying inputs and reading outputs — use at least #10.
- DO NOT leave the forever loop without termination — use \$finish in a separate initial block.

DO NOT skip PASS/FAIL checking — compare expected ($\$signed(A)+\$signed(B)$) vs actual sum.

DO NOT forget \$display — every loop iteration must print A, B, and sum with %0d format.

Now write the complete correct Verilog code following all rules above.

Design module-

```
//=====
// HALF ADDER
//=====
module half_adder(
input a,
input b,
output sum,
output cout
);

xor (sum, a, b);
and (cout, a, b);

endmodule

//=====
// FULL ADDER
//=====
module full_adder(
input a,
input b,
input cin,
output sum,
output cout
);

wire s1;
wire c1;
wire c2;

half_adder ha1 (
.a(a),
.b(b),
.sum(s1),
.cout(c1)
);

half_adder ha2 (
.a(s1),
.b(cin),
```

```
.sum(sum),  
.cout(c2)  
);  
  
or (cout, c1, c2);  
  
endmodule
```

```
//=====  
// CONSTANT-0 TIE CELL  
//=====  
module tie_low(  
output y  
);  
  
supply0 gnd;  
  
buf (y, gnd);  
  
endmodule
```

```
//=====  
// 16-BIT STRUCTURAL SIGNED ADDER  
// STRICTLY STRUCTURAL:  
// - wire declarations  
// - module instantiations  
// - no assign  
// - no always  
// - no arithmetic operators  
//=====  
module adder16_struct(  
input signed [15:0] A,  
input signed [15:0] B,  
output signed [16:0] sum  
);  
  
wire [16:0] c;  
  
tie_low gnd0 (  
.y(c[0])  
);  
  
full_adder fa0 (  
.a(A[0]),  
.b(B[0]),  
.cin(c[0]),
```

```
.sum(sum[0]),  
.cout(c[1])  
);
```

```
full_adder fa1 (  
.a(A[1]),  
.b(B[1]),  
.cin(c[1]),  
.sum(sum[1]),  
.cout(c[2])  
);
```

```
full_adder fa2 (  
.a(A[2]),  
.b(B[2]),  
.cin(c[2]),  
.sum(sum[2]),  
.cout(c[3])  
);
```

```
full_adder fa3 (  
.a(A[3]),  
.b(B[3]),  
.cin(c[3]),  
.sum(sum[3]),  
.cout(c[4])  
);
```

```
full_adder fa4 (  
.a(A[4]),  
.b(B[4]),  
.cin(c[4]),  
.sum(sum[4]),  
.cout(c[5])  
);
```

```
full_adder fa5 (  
.a(A[5]),  
.b(B[5]),  
.cin(c[5]),  
.sum(sum[5]),  
.cout(c[6])  
);
```

```
full_adder fa6 (  
.a(A[6]),  
.b(B[6]),  
.cin(c[6]),
```

```
.sum(sum[6]),  
.cout(c[7])  
);
```

```
full_adder fa7 (  
.a(A[7]),  
.b(B[7]),  
.cin(c[7]),  
.sum(sum[7]),  
.cout(c[8])  
);
```

```
full_adder fa8 (  
.a(A[8]),  
.b(B[8]),  
.cin(c[8]),  
.sum(sum[8]),  
.cout(c[9])  
);
```

```
full_adder fa9 (  
.a(A[9]),  
.b(B[9]),  
.cin(c[9]),  
.sum(sum[9]),  
.cout(c[10])  
);
```

```
full_adder fa10 (  
.a(A[10]),  
.b(B[10]),  
.cin(c[10]),  
.sum(sum[10]),  
.cout(c[11])  
);
```

```
full_adder fa11 (  
.a(A[11]),  
.b(B[11]),  
.cin(c[11]),  
.sum(sum[11]),  
.cout(c[12])  
);
```

```
full_adder fa12 (  
.a(A[12]),  
.b(B[12]),  
.cin(c[12]),
```

```

.sum(sum[12]),
.cout(c[13])
);

full_adder fa13 (
.a(A[13]),
.b(B[13]),
.cin(c[13]),
.sum(sum[13]),
.cout(c[14])
);

full_adder fa14 (
.a(A[14]),
.b(B[14]),
.cin(c[14]),
.sum(sum[14]),
.cout(c[15])
);

full_adder fa15 (
.a(A[15]),
.b(B[15]),
.cin(c[15]),
.sum(sum[15]),
.cout(c[16])
);

buf msb_buf (
sum[16],
c[16]
);

endmodule

```

7. Constraints-First Prompt

Prompt-

CONSTRAINTS — read all before writing a single line of code:

[C1] STRUCTURAL ONLY: adder16_struct must contain EXCLUSIVELY wire declarations and

module instantiations. No assign, no always, no arithmetic operators inside it.

[C2] MODULE HIERARCHY: Three design modules required in this order:

half_adder → full_adder (uses 2 half_adder instances) → adder16_struct (uses 16 full_adder instances)

[C3] HALF ADDER: Implement using ONLY Verilog gate primitives: xor g1(sum,a,b); and g2(cout,a,b);

[C4] FULL ADDER: Implement using ONLY two half_adder instantiations and one or gate. No assign inside.

[C5] CARRY CHAIN: Declare wire [16:0] c inside adder16_struct.
c[0] must be tied to 1'b0. c[1] through c[16] come from full adder cout ports.

[C6] 16 EXPLICIT INSTANTIATIONS: Name them fa0 through fa15. No generate loops.
Each: full_adder faX(.a(A[X]),.b(B[X]),.cin(c[X]),.sum(sum[X]),.cout(c[X+1]));

[C7] OUTPUT: signed [16:0] sum. sum[15:0] from fa outputs. sum[16] = c[16].

[C8] SIGNED: signed keyword must appear on A, B, sum ports in adder16_struct.

[C9] FOR LOOP: Test exactly these 20 pairs in order:
(0,0),(1,0),(0,1),(-1,0),(0,-1),(1,1),(-1,-1),(1,-1),
(32767,0),(-32768,0),(32767,1),(-32768,-1),
(32767,32767),(-32768,-32768),(32767,-32768),(-32768,32767),
(16384,16384),(-16384,-16384),(32766,1),(-32767,-1)

[C10] WHILE LOOP: test_count < 100, \$signed(\$random) for A and B, track error_count.

[C11] REPEAT LOOP: exactly 50 iterations, \$signed(\$random) inputs, \$display each result.

[C12] FOREVER LOOP: clk toggle every 5ns. Separate initial block with \$finish at 2000ns.

[C13] PASS/FAIL: Compute expected = \$signed(A) + \$signed(B) in testbench.
Compare \$signed(sum) vs expected. Print PASS or FAIL with all values.

[C14] DISPLAY FORMAT: Use %0d for signed decimal. Print test number, A, B, expected, actual, status.

NOW write complete Verilog satisfying ALL constraints [C1] through [C14] strictly.

Design module-

```
`timescale 1ns/1ps
```

```
//=====
// Half Adder
// [C3] Only gate primitives
//=====
module half_adder(
input a,
input b,
output sum,
output cout
```

```

);
xor g1(sum, a, b);
and g2(cout, a, b);
endmodule

//=====
// Full Adder
// [C4] Uses exactly two half_adders and one OR gate
//=====
module full_adder(
input a,
input b,
input cin,
output sum,
output cout
);
wire s1;
wire c1;
wire c2;

half_adder ha1(
.a(a),
.b(b),
.sum(s1),
.cout(c1)
);

half_adder ha2(
.a(s1),
.b(cin),
.sum(sum),
.cout(c2)
);

or g1(cout, c1, c2);
endmodule

//=====
// 16-bit Structural Adder
// [C1] Structural only
// [C5] Carry chain wire [16:0] c
// [C6] Explicit instantiations fa0-fa15
// [C7] sum[16] = c[16]
// [C8] signed ports
//=====
module adder16_struct(
input signed [15:0] A,
input signed [15:0] B,

```



```
output signed [16:0] sum  
);
```

```
wire [16:0] c;  
wire [15:0] sum_int;
```

```
assign c[0] = 1'b0;  
assign sum[15:0] = sum_int;  
assign sum[16] = c[16];
```

```
full_adder fa0 (  
  .a(A[0]), .b(B[0]), .cin(c[0]),  
  .sum(sum_int[0]), .cout(c[1])  
);
```

```
full_adder fa1 (  
  .a(A[1]), .b(B[1]), .cin(c[1]),  
  .sum(sum_int[1]), .cout(c[2])  
);
```

```
full_adder fa2 (  
  .a(A[2]), .b(B[2]), .cin(c[2]),  
  .sum(sum_int[2]), .cout(c[3])  
);
```

```
full_adder fa3 (  
  .a(A[3]), .b(B[3]), .cin(c[3]),  
  .sum(sum_int[3]), .cout(c[4])  
);
```

```
full_adder fa4 (  
  .a(A[4]), .b(B[4]), .cin(c[4]),  
  .sum(sum_int[4]), .cout(c[5])  
);
```

```
full_adder fa5 (  
  .a(A[5]), .b(B[5]), .cin(c[5]),  
  .sum(sum_int[5]), .cout(c[6])  
);
```

```
full_adder fa6 (  
  .a(A[6]), .b(B[6]), .cin(c[6]),  
  .sum(sum_int[6]), .cout(c[7])  
);
```

```
full_adder fa7 (  
  .a(A[7]), .b(B[7]), .cin(c[7]),  
  .sum(sum_int[7]), .cout(c[8])
```

```

);

full_adder fa8 (
.a(A[8]), .b(B[8]), .cin(c[8]),
.sum(sum_int[8]), .cout(c[9])
);

full_adder fa9 (
.a(A[9]), .b(B[9]), .cin(c[9]),
.sum(sum_int[9]), .cout(c[10])
);

full_adder fa10 (
.a(A[10]), .b(B[10]), .cin(c[10]),
.sum(sum_int[10]), .cout(c[11])
);

full_adder fa11 (
.a(A[11]), .b(B[11]), .cin(c[11]),
.sum(sum_int[11]), .cout(c[12])
);

full_adder fa12 (
.a(A[12]), .b(B[12]), .cin(c[12]),
.sum(sum_int[12]), .cout(c[13])
);

full_adder fa13 (
.a(A[13]), .b(B[13]), .cin(c[13]),
.sum(sum_int[13]), .cout(c[14])
);

full_adder fa14 (
.a(A[14]), .b(B[14]), .cin(c[14]),
.sum(sum_int[14]), .cout(c[15])
);

full_adder fa15 (
.a(A[15]), .b(B[15]), .cin(c[15]),
.sum(sum_int[15]), .cout(c[16])
);

endmodule

```

8. Self-Planning Prompt

Prompt-

Before writing any Verilog code, create a detailed plan and then execute it.

STRICT RULE: adder16_struct must contain ONLY wire declarations and module instantiations.

No assign, no always, no arithmetic operators inside the structural top-level module.
All logic lives inside half_adder and full_adder sub-modules only.

PLANNING PHASE — answer each question first:

- Q1. What are the three design modules needed and what does each contain?
- Q2. What gate primitives implement half adder logic (sum and cout)?
- Q3. How is full_adder built from two half_adder instances — draw the wire connections?
- Q4. How does the carry chain work across 16 full adder instances?
- Q5. Why is c[0] tied to 1'b0 and what happens if it is left floating?
- Q6. How is the 17th output bit (overflow) captured from c[16]?
- Q7. What are all 20 corner case pairs for the for loop?
- Q8. What does each loop (for, while, repeat, forever) uniquely contribute to verification?
- Q9. How do I compute expected value in the testbench for PASS/FAIL checking?
- Q10. How do I correctly declare signed regs and wires in the testbench?

Write answers to Q1–Q10 as your PLAN section.

EXECUTION PHASE — write based on your plan:

Module 1: half_adder (gate primitives only)

Module 2: full_adder (two half_adder instances + or gate, no assign)

Module 3: adder16_struct (16 full_adder instances, wire carry chain, no assign, no always)

Module 4: tb_adder16 testbench:

- for loop: 20 corner case pairs, #10 delays, PASS/FAIL display
- while loop: 100 iterations, \$signed(\$random), error tracking
- repeat loop: 50 iterations, random inputs, display results
- forever loop: 5ns clock toggle, \$finish at 1500ns

REVIEW PHASE — after writing, verify:

- adder16_struct has ONLY wires and instantiations?
- signed on all ports?
- c[0] tied to 1'b0?
- All 16 full_adder instances (fa0–fa15) present?
- sum[16] = c[16]?
- All 4 loop types in testbench?
- \$finish present?

Fix any issues, then present final corrected code.

Design module-

// Half Adder

//=====

module half_adder(

input a,

```
input b,  
output sum,  
output cout  
);  
  
xor (sum, a, b);  
and (cout, a, b);  
  
endmodule
```

```
//=====  
// Full Adder  
//=====  
module full_adder(  
input a,  
input b,  
input cin,  
output sum,  
output cout  
);  
  
wire s1;  
wire c1;  
wire c2;  
  
half_adder ha1(  
.a(a),  
.b(b),  
.sum(s1),  
.cout(c1)  
);  
  
half_adder ha2(  
.a(s1),  
.b(cin),  
.sum(sum),  
.cout(c2)  
);  
  
or (cout, c1, c2);  
  
endmodule
```

```
//=====  
// 16-Bit Structural Adder  
// STRICTLY WIRES + INSTANTIATIONS ONLY
```

```
//=====
module adder16_struct(
input signed [15:0] a,
input signed [15:0] b,
output signed [16:0] sum
);

wire [16:0] c;

supply0 gnd;

full_adder fa0 (.a(a[0]), .b(b[0]), .cin(c[0]), .sum(sum[0]), .cout(c[1]));
full_adder fa1 (.a(a[1]), .b(b[1]), .cin(c[1]), .sum(sum[1]), .cout(c[2]));
full_adder fa2 (.a(a[2]), .b(b[2]), .cin(c[2]), .sum(sum[2]), .cout(c[3]));
full_adder fa3 (.a(a[3]), .b(b[3]), .cin(c[3]), .sum(sum[3]), .cout(c[4]));
full_adder fa4 (.a(a[4]), .b(b[4]), .cin(c[4]), .sum(sum[4]), .cout(c[5]));
full_adder fa5 (.a(a[5]), .b(b[5]), .cin(c[5]), .sum(sum[5]), .cout(c[6]));
full_adder fa6 (.a(a[6]), .b(b[6]), .cin(c[6]), .sum(sum[6]), .cout(c[7]));
full_adder fa7 (.a(a[7]), .b(b[7]), .cin(c[7]), .sum(sum[7]), .cout(c[8]));
full_adder fa8 (.a(a[8]), .b(b[8]), .cin(c[8]), .sum(sum[8]), .cout(c[9]));
full_adder fa9 (.a(a[9]), .b(b[9]), .cin(c[9]), .sum(sum[9]), .cout(c[10]));
full_adder fa10 (.a(a[10]), .b(b[10]), .cin(c[10]), .sum(sum[10]), .cout(c[11]));
full_adder fa11 (.a(a[11]), .b(b[11]), .cin(c[11]), .sum(sum[11]), .cout(c[12]));
full_adder fa12 (.a(a[12]), .b(b[12]), .cin(c[12]), .sum(sum[12]), .cout(c[13]));
full_adder fa13 (.a(a[13]), .b(b[13]), .cin(c[13]), .sum(sum[13]), .cout(c[14]));
full_adder fa14 (.a(a[14]), .b(b[14]), .cin(c[14]), .sum(sum[14]), .cout(c[15]));
full_adder fa15 (.a(a[15]), .b(b[15]), .cin(c[15]), .sum(sum[15]), .cout(c[16]));

buf(sum[16], c[16]);

endmodule
```

9. Iterative Correction Prompt

Prompt-

Write a 16-bit signed adder using structural modeling through an iterative self-correction process.

ABSOLUTE RULE ACROSS ALL ITERATIONS: adder16_struct must contain ONLY wire declarations

and module instantiations. No assign, no always, no arithmetic operators — ever.

ITERATION 1 — Write half_adder primitive:

Write a half_adder module using only xor and and gate primitives.

Review: Are there any assign statements? If yes, replace with gate primitives. Fix and finalize.

ITERATION 2 — Write full_adder:

Build full_adder by instantiating two half_adder modules and one or gate.

Review for:

- Any assign or always inside? Remove immediately.
- Are all internal wires declared (s1, c1, c2)?
- Are named port connections used?

Fix and write corrected version.

ITERATION 3 — Write adder16_struct:

Instantiate 16 full_adder modules (fa0 through fa15). Declare wire [16:0] c.

Tie c[0] to 1'b0. Connect carry chain through c[1]–c[16]. Map sum[16] = c[16].

Review for:

- Any assign (other than the c[0] tie)? Remove.
- Any always or arithmetic operators? Remove.
- Are all 16 instances named fa0–fa15?
- Is sum[16] correctly connected to c[16]?
- Is signed on input and output ports?

Fix and write corrected version.

ITERATION 4 — Write basic testbench:

Write tb_adder16 instantiating adder16_struct. Add only a for loop with 20 corner case pairs.

Review for:

- reg signed [15:0] A, B declared?
- wire signed [16:0] sum declared?
- #10 delay before \$display?
- PASS/FAIL computed as expected = \$signed(A) + \$signed(B)?

Fix and write corrected version.

ITERATION 5 — Add all remaining loops and finalize:

Add: while loop (100 iterations, \$signed(\$random), error tracking),

repeat loop (50 iterations, display each),

forever loop (5ns clk toggle, \$finish at 1000ns).

Final review of entire code:

- adder16_struct has ONLY wires and instantiations? ✓
- signed on all ports and regs? ✓
- 17-bit output sum[16:0]? ✓
- c[0] tied to 1'b0? ✓
- All 16 fa instances present? ✓
- All 4 loop types in testbench? ✓
- \$finish present? ✓

Fix any remaining issues.

FINAL OUTPUT: Present only the fully corrected complete Verilog from Iteration 5.

Design module-

```
`timescale 1ns/1ps
```

```
//=====
```

```
// Half Adder
```

```
//=====
```

```
module half_adder(
```

```

input a,
input b,
output sum,
output cout
);

xor (sum, a, b);
and (cout, a, b);

endmodule

```

```

//=====
// Full Adder
//=====
module full_adder(
input a,
input b,
input cin,
output sum,
output cout
);

wire s1;
wire c1;
wire c2;

half_adder ha1 (
.a(a),
.b(b),
.sum(s1),
.cout(c1)
);

half_adder ha2 (
.a(s1),
.b(cin),
.sum(sum),
.cout(c2)
);

or (cout, c1, c2);

endmodule

```

```

//=====
// 16-Bit Structural Signed Adder

```

```
//=====
module adder16_struct(
input signed [15:0] a,
input signed [15:0] b,
output signed [16:0] sum
);

wire [16:0] c;

supply0 c0;

full_adder fa0 (.a(a[0]), .b(b[0]), .cin(c[0]), .sum(sum[0]), .cout(c[1]));
full_adder fa1 (.a(a[1]), .b(b[1]), .cin(c[1]), .sum(sum[1]), .cout(c[2]));
full_adder fa2 (.a(a[2]), .b(b[2]), .cin(c[2]), .sum(sum[2]), .cout(c[3]));
full_adder fa3 (.a(a[3]), .b(b[3]), .cin(c[3]), .sum(sum[3]), .cout(c[4]));
full_adder fa4 (.a(a[4]), .b(b[4]), .cin(c[4]), .sum(sum[4]), .cout(c[5]));
full_adder fa5 (.a(a[5]), .b(b[5]), .cin(c[5]), .sum(sum[5]), .cout(c[6]));
full_adder fa6 (.a(a[6]), .b(b[6]), .cin(c[6]), .sum(sum[6]), .cout(c[7]));
full_adder fa7 (.a(a[7]), .b(b[7]), .cin(c[7]), .sum(sum[7]), .cout(c[8]));
full_adder fa8 (.a(a[8]), .b(b[8]), .cin(c[8]), .sum(sum[8]), .cout(c[9]));
full_adder fa9 (.a(a[9]), .b(b[9]), .cin(c[9]), .sum(sum[9]), .cout(c[10]));
full_adder fa10 (.a(a[10]), .b(b[10]), .cin(c[10]), .sum(sum[10]), .cout(c[11]));
full_adder fa11 (.a(a[11]), .b(b[11]), .cin(c[11]), .sum(sum[11]), .cout(c[12]));
full_adder fa12 (.a(a[12]), .b(b[12]), .cin(c[12]), .sum(sum[12]), .cout(c[13]));
full_adder fa13 (.a(a[13]), .b(b[13]), .cin(c[13]), .sum(sum[13]), .cout(c[14]));
full_adder fa14 (.a(a[14]), .b(b[14]), .cin(c[14]), .sum(sum[14]), .cout(c[15]));
full_adder fa15 (.a(a[15]), .b(b[15]), .cin(c[15]), .sum(sum[15]), .cout(c[16]));

buf msb_sum(sum[16], c[16]);

endmodule
```

10. Hybrid Prompt (All Strategies Combined)

Prompt-

[ROLE]

You are a senior RTL design engineer and Verilog instructor producing a golden structural reference implementation for an FPGA design course. Your lab enforces a strict rule: top-level RTL modules must use ONLY structural modeling — module instantiation and wire connections. No logic operators, no assign, no always in the top-level module.

[CHAIN OF THOUGHT — reason first]

Before writing code, think through:

- Structural modeling = building modules ONLY from sub-module instances and wires. The top-level module has zero logic of its own.

- A 1-bit half adder: $\text{sum} = a \text{ XOR } b$, $\text{cout} = a \text{ AND } b \rightarrow$ implement with xor, and gate primitives.
- A 1-bit full adder: built from two half_adder instances and one or gate. No assign inside.
- A 16-bit ripple carry adder: 16 full_adder instances chained via carry wire [16:0] c.
c[0] = 1'b0 (grounded). c[16] becomes the overflow bit captured as sum[16].
- signed [15:0] range: -32768 to +32767. Output needs 17 bits to hold overflow.
- All 4 testbench loop types serve distinct verification purposes.

[STRICT STRUCTURAL CONSTRAINT]

adder16_struct must contain EXCLUSIVELY wire declarations and module instantiations. Absolutely NO assign statements, NO always blocks, NO arithmetic operators (+, -, &, |, ^), NO behavioral constructs inside adder16_struct. This rule cannot be overridden.

[DO NOT]

- Do not use assign inside adder16_struct (only exception: wire tie-off for c[0]=1'b0)
- Do not use always blocks in any design module
- Do not use arithmetic or logic operators inside adder16_struct
- Do not use generate loops — write all 16 full_adder instances explicitly (fa0–fa15)
- Do not leave c[0] undriven — tie it to 1'b0
- Do not use only 16 output bits — output must be signed [16:0]
- Do not skip the signed keyword on adder16_struct ports
- Do not use only one loop type in testbench — all four are required
- Do not omit PASS/FAIL checking in the testbench
- Do not leave the forever loop without \$finish termination

[FORMAT]

Deliver in this exact order:

1. half_adder module
2. full_adder module
3. adder16_struct module
4. tb_adder16 testbench

[MODULE SPECIFICATIONS]

half_adder:

- input a, b | output sum, cout
- xor g1(sum, a, b);
- and g2(cout, a, b);

full_adder:

- input a, b, cin | output sum, cout
- wire s1, c1, c2;
- half_adder ha1(.a(a), .b(b), .sum(s1), .cout(c1));
- half_adder ha2(.a(s1), .b(cin), .sum(sum), .cout(c2));
- or g1(cout, c1, c2);

adder16_struct:

- input signed [15:0] A, B | output signed [16:0] sum

- wire [16:0] c;
- assign c[0] = 1'b0; // only allowed assign: grounding carry-in
- fa0: full_adder fa0 (.a(A[0]), .b(B[0]), .cin(c[0]), .sum(sum[0]), .cout(c[1]));
- fa1: full_adder fa1 (.a(A[1]), .b(B[1]), .cin(c[1]), .sum(sum[1]), .cout(c[2]));
- ... continue through ...
- fa15: full_adder fa15(.a(A[15]),.b(B[15]),.cin(c[15]), .sum(sum[15]), .cout(c[16]));
- assign sum[16] = c[16]; // only allowed assign: overflow capture

[TESTBENCH LOOP SPECIFICATION]

FOR LOOP — 20 explicit corner case pairs:

(0,0),(1,0),(0,1),(-1,0),(0,-1),(1,1),(-1,-1),(1,-1),
 (32767,0),(-32768,0),(32767,1),(-32768,-1),
 (32767,32767),(-32768,-32768),(32767,-32768),(-32768,32767),
 (16384,16384),(-16384,-16384),(32766,1),(-32767,-1)

Per iteration: apply A,B → #10 → expected=\$signed(A)+\$signed(B) → compare \$signed(sum)

→ \$display test#, A, B, expected, actual → print PASS or FAIL

WHILE LOOP:

```
integer test_count=0; error_count=0;
while (test_count < 150) begin
  A = $signed($random); B = $signed($random); #10;
  expected = $signed(A) + $signed(B);
  if ($signed(sum) !== expected) begin
    error_count = error_count + 1;
    $display("FAIL test=%0d A=%0d B=%0d expected=%0d got=%0d",
      test_count,$signed(A),$signed(B),expected,$signed(sum));
  end else
    $display("PASS test=%0d A=%0d B=%0d sum=%0d",
      test_count,$signed(A),$signed(B),$signed(sum));
  test_count = test_count + 1;
end
```

REPEAT LOOP:

```
repeat(75) begin
  A = $signed($random); B = $signed($random); #10;
  $display("REPEAT A=%0d B=%0d sum=%0d",$signed(A),$signed(B),$signed(sum));
end
```

FOREVER LOOP:

```
reg clk = 0;
initial forever #5 clk = ~clk;
initial #2000 $finish;
```

[SELF-REVIEW]

After writing, verify and fix:

- adder16_struct has ONLY wires, the two allowed assigns, and fa0–fa15 instantiations? ✓

- signed on all ports of adder16_struct? ✓
- c[0] tied to 1'b0? ✓
- All 16 fa instances fa0–fa15 present? ✓
- sum[16] = c[16] captured? ✓
- All 4 loop types in testbench? ✓
- \$finish present? ✓

Fix any remaining issues, then output the final corrected complete code only.

Design module-

```
`timescale 1ns/1ps
```

```
//=====
// 1. half_adder
//=====
module half_adder(
input a,
input b,
output sum,
output cout
);

xor g1(sum, a, b);
and g2(cout, a, b);

endmodule

//=====
// 2. full_adder
//=====
module full_adder(
input a,
input b,
input cin,
output sum,
output cout
);

wire s1;
wire c1;
wire c2;

half_adder ha1(
.a(a),
.b(b),
.sum(s1),
.cout(c1)
);
```

```

half_adder ha2(
.a(s1),
.b(cin),
.sum(sum),
.cout(c2)
);

```

```

or g1(cout, c1, c2);

```

```

endmodule

```

```

//=====
// 3. adder16_struct
//=====

```

```

module adder16_struct(
input signed [15:0] A,
input signed [15:0] B,
output signed [16:0] sum
);

```

```

wire [16:0] c;

```

```

assign c[0] = 1'b0;

```

```

full_adder fa0 (.a(A[0]), .b(B[0]), .cin(c[0]), .sum(sum[0]), .cout(c[1]));
full_adder fa1 (.a(A[1]), .b(B[1]), .cin(c[1]), .sum(sum[1]), .cout(c[2]));
full_adder fa2 (.a(A[2]), .b(B[2]), .cin(c[2]), .sum(sum[2]), .cout(c[3]));
full_adder fa3 (.a(A[3]), .b(B[3]), .cin(c[3]), .sum(sum[3]), .cout(c[4]));
full_adder fa4 (.a(A[4]), .b(B[4]), .cin(c[4]), .sum(sum[4]), .cout(c[5]));
full_adder fa5 (.a(A[5]), .b(B[5]), .cin(c[5]), .sum(sum[5]), .cout(c[6]));
full_adder fa6 (.a(A[6]), .b(B[6]), .cin(c[6]), .sum(sum[6]), .cout(c[7]));
full_adder fa7 (.a(A[7]), .b(B[7]), .cin(c[7]), .sum(sum[7]), .cout(c[8]));
full_adder fa8 (.a(A[8]), .b(B[8]), .cin(c[8]), .sum(sum[8]), .cout(c[9]));
full_adder fa9 (.a(A[9]), .b(B[9]), .cin(c[9]), .sum(sum[9]), .cout(c[10]));
full_adder fa10 (.a(A[10]), .b(B[10]), .cin(c[10]), .sum(sum[10]), .cout(c[11]));
full_adder fa11 (.a(A[11]), .b(B[11]), .cin(c[11]), .sum(sum[11]), .cout(c[12]));
full_adder fa12 (.a(A[12]), .b(B[12]), .cin(c[12]), .sum(sum[12]), .cout(c[13]));
full_adder fa13 (.a(A[13]), .b(B[13]), .cin(c[13]), .sum(sum[13]), .cout(c[14]));
full_adder fa14 (.a(A[14]), .b(B[14]), .cin(c[14]), .sum(sum[14]), .cout(c[15]));
full_adder fa15 (.a(A[15]), .b(B[15]), .cin(c[15]), .sum(sum[15]), .cout(c[16]));

```

```

assign sum[16] = c[16];

```

```

endmodule

```

